



TEMA 02 - TypeScript y Angular

RA 2, RA 4 [FRAMEWORKS] Y RA 7 [DWEC]

Parte 02

EJERCICIO 1:.....	1
EJERCICIO 2:.....	3
EJERCICIO 3:.....	12

EJERCICIO 1:

Crea una carpeta con nombre ejercicio1_apellido1 dentro de fw_t02p02_apellido1_nombre1.

Clona el siguiente repositorio: https://github.com/igijon/angular19_bases.git

Proyecto de Angular de iniciación con la versión 19.

Se trata de un proyecto de la profesora I. Gijón del CFP Virgen de Gracia de Puertollano.

Analiza la estructura y el código del mismo entendiendo sus componentes.

- ¿Qué dependencias adicionales se instalan?
- ¿Es standalone o no?
- ¿Cuántos módulos hay definidos? ¿Dónde se centraliza?
- ¿De qué se encarga AppModule?
- ¿Se definen rutas en AppModule?
- ¿Qué componentes tiene AppModule?
- ¿De qué se encarga DbzModule?
- ¿Se definen rutas en DbzModule?
- ¿Qué componentes tiene DbzModule?
- ¿Qué servicios define DbzModule?
- ¿Qué papel desempeña interfaces y pages en DbzModule?
- ¿De qué se encarga HeroesModule?
- ¿Se definen rutas en HeroesModule?
- ¿Qué componentes tiene HeroesModule?
- ¿De qué se encarga CounterModule?
- ¿Se definen rutas en CounterModule?
- ¿Qué componentes tiene DbzModule?
- ¿A qué crees que se debe esta estructura?



EJERCICIO 2:

El objetivo de esta actividad es llevar a cabo una aplicación real sencilla donde pongamos en práctica estos conceptos básicos que hemos visto en Angular e investiguemos de forma autónoma algunos conceptos más avanzados como el consumo de datos asíncronos desde una API Rest y el uso de almacenamiento local con Localstorage.

Para ello, vamos a construir una aplicación en Angular (usando los estilos predefinidos de bootstrap) que permita buscar capítulos en la API de Rick y Morty: <https://rickandmortyapi.com/>

Funcionalidades principales de la app en la versión 1:

- Página de inicio: Presentación de la aplicación.
- Página de listado de personajes: Mostrará la lista de personajes (preferiblemente con paginación).
- Página de listado de episodios: Mostrará la lista de episodios (preferiblemente con paginación).
- Página de buscador de localizaciones dada una especie: Buscará en qué localizaciones reside una especie y los resultados se mostrarán en formato tabla. Esto lo hará consultando las especies de los residentes en una localización.
- Página de error 404: Se mostrará si el usuario accede a una ruta inexistente.

VERSIÓN 1

Para ello vamos a llevar a cabo los siguientes pasos:

- a. Crea un proyecto en angular de tipo standalone false con nombre FUERA de la carpeta fw_t02p02_apellido1_nombre1: ejercicio2_m_rym_apellido1

```
ng new ejercicio2_m_rym_apellido1 --standalone false
```

Con CSS, sin SSR y sin agente IA.

Inicia Git en el proyecto (si no está ya iniciado).

Adapta brevemente el readme.md que genera ng con tus datos personales.

Comprueba que el fichero .gitignore que genera ng está adaptado a tus necesidades.

Descarga [la siguiente licencia GNU General Public License](#), incorpórala a tu directorio de trabajo.

Haz un commit con esos cambios.

Crea un repositorio con el mismo nombre en GitHub e invítame como colaborador. El repositorio será público y no tendrá ni readme, ni .gitignore ni licencia.

Vincula el repositorio local con GitHub.

```
git remote add origin https://github.com/tuusuario/tu_repo.git
git branch -M main #Si tu repositorio local usa master en vez de main
git push -u origin main
```

b. Crea 5 ramas adicionales a la rama main con estos nombres:

- v1-basic-app

Aunque estas no hay que crearlas hasta que llegue el momento:

- v2-usability
- v3-authentication
- v4-localstorage
- v5-api-laravel

Estas ramas representan versiones importantes de la aplicación. La rama main será la rama estable donde se integrarán los cambios finales de cada versión. Cada vez que una versión esté lista, se hará merge con main.

No trabajes directamente en main, solo debe recibir cambios mediante merge desde las otras ramas.

Muévete a la rama v1-basic-app para empezar.

- c. Instala bootstrap usando npm e incorporarlo al proyecto, tanto a nivel de CCS como de JS.
- d. AppComponent solo debe encargarse de inicializar la aplicación. Se limita a incluir el LayoutComponent.

En cambio,

- AppRoutingModule: Es el módulo encargado de definir y gestionar todas las rutas principales de la aplicación. Aquí se configuran las rutas de cada módulo y las redirecciones necesarias.
 - AppModule: Es el módulo principal de la aplicación. Su función es importar y organizar los módulos clave, como CoreModule, PagesModule y RickMortyModule. También configura los servicios globales y el sistema de enrutamiento.
- e. Establece la siguiente estructura de módulos y componentes, **así como sus interacciones** (haz los imports necesario al AppModule y entre módulos). No es necesario generar los ficheros .spec.
- Módulo **core**
 - Se encarga de los componentes compartidos que forman la estructura de la app.
 - Todo los componentes del módulo core irá en un directorio con nombre

'core/components':

- LayoutComponent → Estructura el diseño de la aplicación y contener los componentes globales. Aquí irá el también el main y el <router-outlet> (dentro del main)
 - HeaderComponent → Contiene el título y el logo.
 - MenuComponent → Contiene el menú de navegación.
 - BreadcrumbComponent → Migas de pan
 - FooterComponent → Pie de página.
 - Aquellos componentes adicionales que consideres oportunos.
- También puede incluir servicios globales si los hubiera en 'core/services'.
- Este módulo NO debe usar Lazy Loading ya que sus componentes son usados en toda la aplicación.

Ejemplo de creación:

```
ng g m core
ng g c core/components/header --standalone false
```

- Módulo **rickmorty**

- Todo los componentes del módulo rickmorty irá en un directorio con nombre 'rickmorty/components':
 - CharactersComponent → Página con la lista de personajes.
 - En primer lugar se mostrarán 6 al azar.
 - EpisodesComponent → Listado de episodios.
 - En primer lugar se mostrarán 6 al azar.
 - SearchLocationComponent → Tabla con localizaciones en las que reside una especie introducida por el usuario en un buscador.
 - Aquellos componentes adicionales que consideres oportunos.
- Todo los servicios del módulo core irá en un directorio con nombre 'rickmorty/services':
 - CharacterService → Servicio para obtener personajes.
 - EpisodesService → Servicio para obtener episodios
 - SearchLocationService → Servicio para obtener localizaciones.
 - Aquellos servicios adicionales que consideres oportunos.
- Este módulo debe usar Lazy Loading para optimizar la carga de la app. Se explica más abajo.

- Módulo **pages**

- Contiene las páginas principales de la aplicación.
- Todo los componentes del módulo pages irá en un directorio con nombre 'pages/components':
 - HomeComponent → Página principal.
 - NotFoundComponent → Página de error 404 si la URL no existe.

- Aquellos componentes adicionales que consideres oportunos.
- Este módulo debe usar Lazy Loading para optimizar la carga de la app. Se explica más abajo.

Nota: Convenciones: Angular sigue la convención de Entidad+Tipo, lo cual hace que el código sea más intuitivo y uniforme.

f. Define el sistema de rutas.

Para optimizar la velocidad de carga, utilizaremos Lazy Loading (Carga Diferida) en los módulos que lo necesiten. Esto significa que en lugar de importar los módulos directamente en AppModule, los cargaremos solo cuando el usuario los necesite. En Angular, solo tenemos que definir las rutas usando loadChildren para aplicar Lazy Loading. Para estructurar bien nuestra aplicación, utilizaremos Lazy Loading.

Módulos que deben usar Lazy Loading:

- PagesModule (Módulo de Páginas Generales).
- RickMortyModule (Módulo de Personajes y API)

El CoreModule NO debe usar Lazy Loading, ya que contiene componentes comunes que siempre deben estar disponibles en toda la aplicación desde el inicio.

En el archivo app-routing.module.ts define las rutas principales de la aplicación y carga los módulos de forma diferida con loadChildren.

En pages.module.ts y rickmorty.module.ts, configura sus rutas internas usando RouterModule.forChild().

Asegúrate de que los módulos no estén importados en app.module.ts, sino que solo se carguen a través de las rutas.

g. Vamos a utilizar Bootstrap para diseñar las vistas. Tenemos que crear un diseño responsivo y atractivo.

Además, debes utilizar componentes clave como:

- Navbar → Para la navegación principal de la aplicación. Tendrá un logo de Rick & Morty. Y de momento solo los enlaces a Home, Personajes y Capítulos.
- Header & Footer → Encabezado y pie de página reutilizables.
- Jumbotron → Para resaltar información importante en la página principal. Tendrá una breve descripción de la aplicación y una imagen de Rick & Morty.
- Card → Para mostrar información de los personajes y capítulos de manera ordenada.
 - Para personajes incluirá:
 - Id

- Nombre
- Especie
- Género
- Estado (Vivo/Muerto/Desconocido)
- Imagen
- Para capítulos incluirá:
 - Id
 - Nombre del episodio
 - Número del episodio
 - La lista de personajes.

Nota: Las imágenes se optimizarán con NgOptimizedImage.

- Collapse → Para mostrar/ocultar detalles adicionales de los personajes que aparecen en los capítulos.
- h. Para estructurar correctamente los datos que obtenemos de la API de Rick & Morty, utilizaremos interfaces en TypeScript. Las interfaces nos permiten definir la estructura de los datos y evitar el uso de any, lo que hace que el código sea más legible, seguro y fácil de mantener.

Hazlo dentro de la carpeta interfaces dentro del módulo rickmorty.

Define la interfaz Character para representar los personajes de la API, define la interfaz Episode para representar los episodios y define la interfaz Location para representar las localizaciones.
- i. Incorpora el uso de algún filtro personalizado (Pipes) en el módulo rickmorty.

VERSIÓN 2

- Muévete a la rama v2-usability.
- Mejora la aplicación para que las páginas de personajes y capítulos soporten paginación de 6 en 6 tarjetas.

VERSIÓN 3

- Muévete a la rama v3-authentication
- Implementa autenticación para que la aplicación distinga en parte pública (página Home) y parte privada (resto de páginas).
 - Usa guards.
 - Si el usuario no tiene acceso, la aplicación tiene que redirigir automáticamente a la página de inicio (HomeComponent). No es necesario crear una página que nos explique que estamos intentando acceder a una zona restringida.

Para ello, sigue las siguientes indicaciones.

- c. Este apartado lo vamos a hacer usando sessionStorage (método más fácil y común en apps sin requerimientos de alta seguridad). Esta implementación es para pruebas. En un entorno real, la autenticación se haría con un servidor seguro y tokens JWT.
- d. En el módulo **pages** crea un componente que se llame LoginComponent que contenga un formulario de login para que los usuarios inicien sesión. El registro no se contempla de momento. El inicio de sesión se hace con email/contraseña. Hay que validar que el campo es un email válido y que la contraseña tiene al menos 4 caracteres. No se permiten los campos vacíos.
- e. Crear un nuevo módulo con nombre **auth**, ya que la autenticación es una funcionalidad separada que no debe mezclarse con otros módulos.

Dentro de ese módulo y manteniendo la estructura de directorios que hemos usando crea:

- UserRole (enumerado) → Define los roles de usuario (**admin**, **user**, **guest**).
 - User (interfaz) → Define la estructura del usuario autenticado (id, name, email y role).
 - AuthService → Servicio que maneja la autenticación (*llamada a la "API", gestión del usuario autenticado, logout, etc.*).
 - Si las credenciales son correctas, el servicio devuelve un json con el objeto usuario (con los atributos id, name, email y role). Esto se pasa a una función que guarda el objeto en sessionStorage. Después se redirige al Home sabiendo que estamos autenticados.
 - Si son incorrectas, se devuelve un json con un error 401 y el mensaje "Usuario o contraseña incorrectos" (json con los atributos error y message). Se nos muestra un mensaje.
 - AuthGuard → Guard para restringir el acceso a ciertas rutas. Revisa si el usuario está autenticado para permitir el acceso a rutas protegidas.
 - Otras utilidades si fueran necesarias.
- f.  **IMPORTANTE:** Dado que no contamos con un servidor real, usaremos un JSON estático para almacenar los usuarios y simular una API. Esto es peligroso en un entorno real, pero sirve para pruebas locales.

Descarga el [siguiente json](#) y guárdalo en la carpeta "auth/simulateapi". Las contraseñas son los nombres de usuario y se ha usado **bcryptjs**.

El AuthService debe simular una llamada HTTP que obtiene el JSON y compara el usuario y la contraseña ingresados.

- g. En app-routing.module.ts usa AuthGuard para proteger las rutas privadas y redirigir a Home en caso de no estar autenticado

- h. Adapta el menú de navegación para que detecte que hemos iniciado sesión (o no) y para que nos permita cerrarla en los casos pertinentes.
- i. De momento, no es necesario avisar del uso de cookies.

VERSIÓN 4

En esta versión, vamos a permitir que los usuarios autenticados puedan guardar capítulos como favoritos (junto con una calificación de 1 a 5 obligatoria y una observación obligatoria del mismo) y gestionarlos desde una nueva sección en la aplicación. Para ello, implementaremos un nuevo módulo y utilizaremos LocalStorage para almacenar los datos.

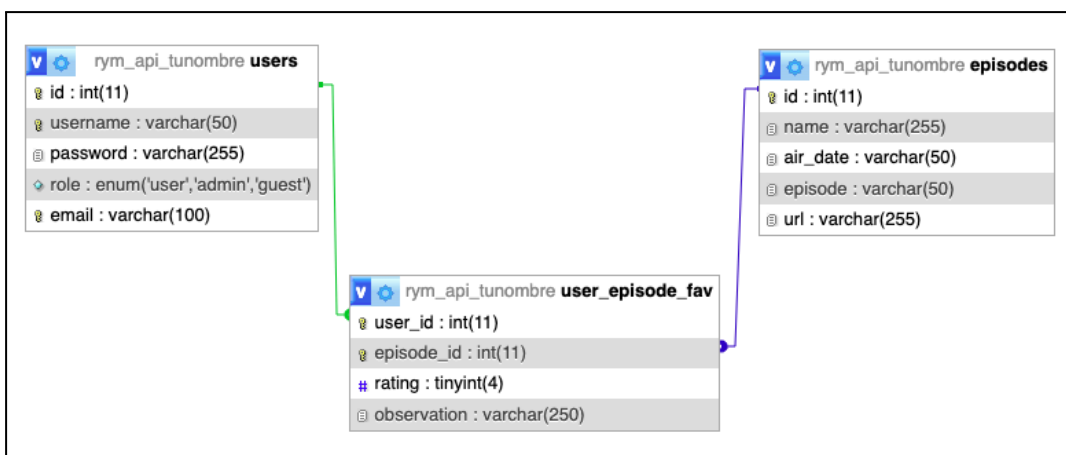
- a. Muévete a la rama v4-localstorage.
- b. El componente EpisodesComponent debe detectar si el usuario está autenticado (consultando sessionStorage). Si el usuario está logueado, debe mostrar un botón para "Guardar en Favoritos" o "Eliminar de Favoritos", dependiendo de si el episodio ya está guardado como favorito o no. La gestión de favoritos se realizará consultando LocalStorage. Un usuario no puede guardar el mismo episodio dos veces.
- c. Crea un módulo nuevo llamado **myrickmorty**. Este módulo gestionará la funcionalidad de favoritos. Dentro de ese módulo y manteniendo la estructura de directorios que hemos usando crea:
 - Componentes:
 - **FavoritesComponent**: Página donde se listan los capítulos guardados como favoritos.
 - Los capítulos se guardarán en LocalStorage, asociándolos al id del usuario autenticado.
 - De cada capítulo se guarda su id, name, air_date, episode y url.
 - Además es obligatorio añadir y guardar:
 - Valoración del usuario (número entre 1 y 5)
 - Observación (comentario personal del usuario)Las observaciones no podrán editarse una vez guardadas. Ni las puntuaciones.
 - Se mostrarán todas las tarjetas de capítulos guardadas sin paginación.
 - Un capítulo se podrá eliminar tanto desde FavoritesComponent como desde EpisodesComponent.
 - Validaciones:
 - Un episodio no puede guardarse sin una puntuación y una observación.
 - Solo se aceptan valores enteros entre **1 y 5**.
 - Una observación no puede estar vacía ni superar los 250 caracteres.

- Un usuario no puede guardar el mismo episodio dos veces.
 - Servicios
 - **FavoritesService:** Servicio encargado de gestionar el almacenamiento y recuperación de capítulos favoritos en LocalStorage.
- ⚠ IMPORTANTE:** Esta implementación se realiza con LocalStorage aunque, los favoritos deberían almacenarse en una base de datos y gestionarse mediante una API REST. Sin embargo, para simplificar este ejercicio, no implementaremos la persistencia en el servidor.
- d. Añade una opción "Favoritos" en el menú, visible solo para usuarios autenticados y define las rutas necesarias para que se pueda acceder a la página de favoritos desde la aplicación.

VERSIÓN 5 API EN LARAVEL

- a. Siguiendo las indicaciones dadas en DWES desarrolla usando Laravel una API que sea consumida por nuestra aplicación y que nos permita resolver:
- La autenticación de los usuario.
 - La gestión de favoritos.
- b. La base de datos debe cumplir los siguientes requisitos:
- Se llamará: `rym_api_apellido1`
 - Existirá un usuario que se llamará `rym_user` con contraseña `2025_rYm`:

```
CREATE USER 'rym_user'@'localhost' IDENTIFIED BY '2025_rYm';
GRANT ALL PRIVILEGES ON rym_api_tuapellido.* TO
'rym_user'@'localhost';
FLUSH PRIVILEGES;
```
 - Descarga el [siguiente fichero .mysql](#), edita el nombre de la base de datos con tu nombre y el USE. Guarda los cambios e importa ese fichero en tu SGBD.
 - La base de datos tiene el siguiente esquema:



- Debe haber un fichero .env

.env

```
DB_HOST=localhost (o 127.0.0.1 en un entorno Windows)
DB_USER=rym_user
DB_PASS=2025_rYm
DB_NAME=rym_api_tuapellido
```

- c. Los endpoints pueden ser (aunque queda a tu criterio):

POST	/api/login	-> Autenticación y generación de token
POST	/api/logout	-> Cierre de sesión (invalida el JWT)
GET	/api/user	-> Devuelve los datos del usuario autent.
GET	/api/favorites	-> Lista los capítulos favoritos del usu.
POST	/api/favorites	-> Agrega un capítulo a favoritos
DELETE	/api/favorites/{id}	-> Elimina un capítulo de favoritos

- d. La autenticación de los usuarios:

- Si las credenciales son correctas, el servicio devuelve un json con el objeto usuario (con los atributos name, email y role).
- Si son incorrectas, la API debería devolver un error 401, y el servicio manejarlo mostrando un mensaje "Usuario o contraseña incorrectos" (json con los atributos error y message)
- Si hay un error inesperado, la API devuelve un 500, y el servicio muestra un mensaje de error general "Error en el servidor, intenta más tarde" (json con los atributos error y message).

- e. La autenticación se hará usando JWT.

- Configurar el tiempo de expiración del token JWT para mejorar la seguridad.

- f. Permite que la API devuelva los capítulos favoritos de cada usuario. Además permitirá eliminar capítulos favoritos o añadir nuevos.
 - Todas las respuestas JSON deberían seguir una estructura clara.
- g. Adapta tu aplicación en Angular usando JWT.
- h. Adapta tu aplicación en Angular para hacer uso de esta API.

EJERCICIO 3:

En este ejercicio vamos a adaptar el ejercicio realizado con TS sobre recetas a Angular. Para ello, realiza las siguientes tareas.

- AMPLIACIÓN SEGUNDA ORDINARIA -

Para superar la prueba de recuperación, será obligatorio que los distintos requisitos funcionales y las diferentes opciones del menú estén desarrollados y funcionando, al menos, al 50%. Si alguna de las partes principales de la práctica se deja sin desarrollar o presenta un grado de desarrollo insuficiente, la práctica se considerará no superada.

Antes de empezar: Todas estas ampliaciones y modificaciones deberán realizarse respetando los criterios seguidos durante el desarrollo de la práctica, especialmente en lo relativo al modelo de datos, la validación, la gestión de errores, la manipulación del DOM y la organización general de la aplicación.

Mismas ampliaciones que en TS: [02_Frameworks_Tema02_Parte01_ejercicios_amg](#)

Normativa: La entrega se hará usando el repositorio compartido realizando un push con la solución final antes de la fecha límite de entrega. El mensaje del último commit debe ser: "Recuperación 2a. Ordinaria".

- a. Crea un proyecto en angular de tipo **standalone true** con nombre **DENTRO de la carpeta** `fw_t02p02_apellido1_nombre1`: `ejercicio3_meals_apellido1`

```
ng new ejercicio3_meals_apellido1
```

Con CSS, sin SSR y sin agente IA.

El comando anterior habrá detectado que este proyecto se ha creado dentro de un repositorio superior y por tanto no habrá iniciado git.

Adapta brevemente el `readme.md` que genera ng con tus datos personales.

Ejercicio3MealsApellido1

This project was generated using Angular CLI version 21.1.0.

About

Nombre y Apellidos

Curso

Descripción del proyecto

Development server

To start a local development server, run:

Comprueba que el fichero `.gitignore` que genera ng está adaptado a tus necesidades.

Descarga [la siguiente licencia GNU General Public License](#), incorpórala a tu directorio de trabajo.

Haz un commit con esos cambios.

- b. Instala bootstrap usando npm e incorporarlo al proyecto, tanto a nivel de CCS como de JS.

```
npm install bootstrap
```

```
//Instalamos también los iconos  
npm install bootstrap-icons
```

Ve al archivo `angular.json` y busca la sección "styles" y "scripts" dentro de "build":

```
"styles": [  
  "node_modules/bootstrap/dist/css/bootstrap.min.css",  
  "node_modules/bootstrap-icons/font/bootstrap-icons.css",  
  "src/styles.css"  
],
```

```
"scripts": [  
  "node_modules/bootstrap/dist/js/bootstrap.bundle.min.js"  
]
```

Nota: También puede existir la sección "styles" y "scripts" dentro de "test". **TEN CUIDADO.**

- c. Recuerda que en este proyecto de Angular no se trabajará con NgModules y que toda la aplicación se estructura a partir de componentes standalone, rutas y configuración global.
- app.config.ts: Define los providers globales (router, http, etc.).
 - app.routes.ts: Contiene todas las rutas de la aplicación.
 - app.ts: Es el componente raíz. Su función principal es cargar el layout general. De momento borra todo el contenido de app.html menos: `<router-outlet />`

- d. Crea la carpeta c_layout en src/app

Todos los componentes de este apartado que vamos a crear dentro de esta carpeta deben:

- Ser standalone
- Crearse con fichero .spec.ts
- Estar pensados para reutilizar estructura, no contenido de páginas

Crea los siguientes componentes:

```
ng generate component c_layout/footer
```

```
footer.html  
  
<footer class="bg-light text-center text-muted py-3 mt-auto border-top">  
  <div class="container">  
    <small>  
      DWEC & Frameworks | Nombre Apellido1 | IES Gregorio Prieto  
    </small>  
  </div>  
</footer>
```

```
ng generate component c_layout/login-widget
```

- No es la página de login, sino un componente reutilizable que:
 - a. Muestra botones de Login / Registro o
 - b. Permite cerrar sesión si el usuario está autenticado
- Este componente puedes colocarlo en el header o en el menú de navegación .

```
login-widget.ts  
  
import { Component } from '@angular/core';  
import { RouterLink } from '@angular/router';
```

```
@Component({
  selector: 'app-login-widget',
  imports: [RouterLink],
  templateUrl: './login-widget.html',
  styleUrls: ['./login-widget.css'],
})
export class LoginWidget {}
```

login-widget.html

```
<a routerLink="/login" class="btn btn-outline-light btn-sm d-flex
align-items-center gap-2">
  <i class="bi bi-person-circle"></i>
  <span>Login</span>
</a>
```

ng generate component c layout/navbar

navbar.ts

```
import { Component } from '@angular/core';
import { RouterLink, RouterLinkActive } from '@angular/router';
@Component({
  selector: 'app-navbar',
  imports: [RouterLink, RouterLinkActive],
  templateUrl: './navbar.html',
  styleUrls: ['./navbar.css'],
})
export class Navbar {}
```

navbar.html

```
<nav class="navbar navbar-expand navbar-light bg-light border-bottom">
  <div class="container">
    <ul class="navbar-nav">
      <li class="nav-item">
        <a class="nav-link"
          routerLink="/"
          routerLinkActive="active"
          [routerLinkActiveOptions]="{ exact: true }">
          Home
        </a>
      </li>
    </ul>
  </div>
</nav>
```

ng generate component c layout/header

header.ts

```
import { Component } from '@angular/core';
import { LoginWidget } from '../login-widget/login-widget';
```

```
@Component({
  selector: 'app-header',
  imports: [LoginWidget],
  templateUrl: './header.html',
  styleUrls: ['./header.css'],
})
export class Header {}
```

header.html

```
<header class="bg-dark text-white py-3">
  <div class="container d-flex justify-content-between
align-items-center">
    <h1 class="h4 mb-0">Angular Meals App</h1>
    <app-login-widget></app-login-widget>
  </div>
</header>
```

ng generate component c layout/layout

layout.ts

```
import { Component } from '@angular/core';
import { Header } from '../header/header';
import { Navbar } from '../navbar/navbar';
import { RouterOutlet } from '@angular/router';
import { Footer } from '../footer/footer';

@Component({
  selector: 'app-layout',
  imports: [Header, Navbar, RouterOutlet, Footer],
  templateUrl: './layout.html',
  styleUrls: ['./layout.css'],
})
export class Layout {}
```

layout.html

```
<app-header></app-header>
<app-navbar></app-navbar>

<main class="container my-4">
  <router-outlet></router-outlet>
</main>

<app-footer></app-footer>
```

Actualiza las rutas:

app.routes.ts

```
import { Routes } from '@angular/router';
import { Layout } from '../c_layout/layout/layout';
export const routes: Routes = [
  {
```



```
path: '',
component: Layout,
children: [],
},
];
```

e. Crea la carpeta `c_pages` en `src/app`

Todos los componentes de este apartado que vamos a crear dentro de esta carpeta deben:

- Ser standalone
- Crearse con fichero `.spec.ts`
- Representan páginas/vistas completas que se cargan por rutas

Crea los siguientes componentes:

```
ng generate component c_pages/not-found
```

not-found.ts

```
import { Component } from '@angular/core';
import { RouterLink } from '@angular/router';
@Component({
  selector: 'app-not-found',
  imports: [RouterLink],
  templateUrl: './not-found.html',
  styleUrls: ['./not-found.css'],
})
export class NotFound {}
```

not-found.html

```
<div class="container d-flex justify-content-center align-items-center
vh-100">
  <div class="text-center">
    <h1 class="display-4 text-danger">404</h1>
    <h2 class="mb-3">Página no encontrada</h2>
    <p class="text-muted mb-4">
      La URL que has introducido no existe o no está disponible.
    </p>
    <a routerLink="/" class="btn btn-primary">
      Volver al inicio
    </a>
  </div>
</div>
```

```
ng generate component c_pages/meals-category
```

```
ng generate component c_pages/meals-save
```

```
ng generate component c_pages/home
```

(usa `meals-category` y `meals-save`)

home.ts

```
import { Component } from '@angular/core';
import { MealsCategory } from '../meals-category/meals-category';
import { MealsSave } from '../meals-save/meals-save';

@Component({
  selector: 'app-home',
  imports: [MealsCategory, MealsSave],
  templateUrl: './home.html',
  styleUrls: ['./home.css'],
})
export class Home {
  public isAuthenticated = false; // más adelante vendrá de un AuthService
}
```

home.html

```
<app-meals-category></app-meals-category>

@if (isAuthenticated) {
  <app-meals-save></app-meals-save>
}
```

```
ng generate component c_pages/details-meal
ng generate component c_pages/details-save
ng generate component c_pages/details
(usa details-meal y details-save)
```

```
ng generate component c_pages/plan-week-create
ng generate component c_pages/plan-week-list
ng generate component c_pages/plan-week
(usa details-meal y details-save)
```

```
ng generate component c_pages/login
```

login.html - Solo visual

```
<div class="container d-flex justify-content-center align-items-center
vh-100">
  <div class="card shadow-sm" style="max-width: 420px; width: 100%;">

    <div class="card-body">

      <!-- Tabs -->
      <ul class="nav nav-tabs nav-fill mb-4" role="tablist">
        <li class="nav-item" role="presentation">
          <button class="nav-link active" id="login-tab"
data-bs-toggle="tab" data-bs-target="#login" type="button"
role="tab">
```

```
        Login
      </button>
    </li>

    <li class="nav-item" role="presentation">
      <button class="nav-link" id="register-tab" data-bs-toggle="tab"
data-bs-target="#register" type="button"
        role="tab">
        Registro
      </button>
    </li>
  </ul>

  <!-- Tab content -->
  <div class="tab-content">

    <!-- LOGIN -->
    <div class="tab-pane fade show active" id="login" role="tabpanel">

      <form>
        <div class="mb-3">
          <label for="email" class="form-label">Correo
electrónico</label>
          <input type="email" id="email" name="email"
class="form-control"
            placeholder="usuario@educastillalamancha.es">
        </div>

        <div class="mb-4">
          <label for="password" class="form-label">Contraseña</label>
          <input type="password" id="password" name="password"
class="form-control" placeholder="*****">
        </div>

        <div class="d-grid">
          <button type="submit" class="btn btn-primary">
            Entrar
          </button>
        </div>
      </form>
    </div>

    <!-- REGISTER -->
    <div class="tab-pane fade text-center" id="register"
role="tabpanel">

      <p class="text-muted mt-4">
        En breve.
      </p>
    </div>

  </div>
</div>
</div>
</div>
```

Actualiza las rutas:

```
app.routes.ts

import { Routes } from '@angular/router';
import { Layout } from '../c_layout/layout/layout';
import { Home } from '../c_pages/home/home';
import { Details } from '../c_pages/details/details';
import { PlanWeek } from '../c_pages/plan-week/plan-week';
import { Login } from '../c_pages/login/login';
import { NotFound } from '../c_pages/not-found/not-found';
export const routes: Routes = [
  //Con layout
  {
    path: '',
    component: Layout,
    children: [
      { path: '', component: Home },
      { path: 'details/:id', component: Details },
      { path: 'plan-week', component: PlanWeek },
    ],
  },
  //Sin layout
  { path: 'login', component: Login },
  { path: '**', component: NotFound },
];
```

f. Crea la carpeta model en src/app

Dentro de model, crea las siguientes interfaces:

```
ng generate interface model/i-user
ng generate interface model/i-user-meal
ng generate interface model/i-weekly-plan
ng generate interface model/i-weekly-plan-day
ng generate interface model/i-user-mini-meal
ng generate interface model/i-my-meal
ng generate interface model/i-category (opcional)
ng generate interface model/i-ingr-measure (opcional)
```

Recuerda: Las interfaces se utilizan únicamente para definir la estructura de los datos. No contienen lógica ni métodos, solo propiedades.

Crea las siguientes clases dentro de model:

```
ng generate class model/auth-session
    También puede ser una interfaz.
```

```
ng generate class model/util
```

g. Crea la carpeta services en src/app

Dentro de services, crea las siguientes servicios:

```
ng generate service services/api-service
ng generate service services/local-storage-service
ng generate service services/auth-service
```

AuthService será el servicio responsable de gestionar la autenticación del usuario en la aplicación. No representa datos, gestiona estado y comportamiento:

- iniciar sesión
- cerrar sesión
- comprobar si hay un usuario autenticado
- recuperar la sesión almacenada

Usa StorageService y nos va a facilitar mucho el uso de Guards.

AuthSession será la clase que representa una sesión autenticada. También puede ser una interfaz ahora que pasamos toda la lógica a AuthService.

- No contiene lógica de aplicación
- No accede a localStorage

- h. Una vez montada la estructura del proyecto, el siguiente paso es adaptar tu solución en TypeScript puro de tema anterior a Angular, respetando la arquitectura del proyecto (componentes standalone, routing, servicios y modelos).

Nota: Si necesitas crear algún otro elemento, puedes crearlo.

- i. Aspectos a tener en cuenta:

- Obligatorio: Todas las imágenes de recetas deben cargarse usando ngSrc (no src) para que Angular gestione la carga.
- Obligatorio: Al menos uno de los formularios tiene que validarse usando Reactive Forms y Bootstrap. Un ejemplo: [Anexo_Frameworks_Tema02_Parte02_25-26_ejercicios_amg](#)
- Obligatorio: Usar Guards de autenticación.
- Obligatorio: En Home, la sección donde se muestran las últimas 4 recetas guardadas puede cargarse con @defer para mejorar rendimiento.
- Obligatorio: Los errores deberán mostrarse en la vista con componentes Bootstrap (alert-danger, alert-warning, alert-info). Aunque ya se debería haber estado haciendo así.
- Voluntario: Crea un Pipe ingredientsCount que cuente cuántos ingredientes distintos tiene una receta.
- Voluntario: Si tienes que realizar alguna petición AJAX nueva, puedes probar HttpClient



- Voluntario: Configura la ruta /plan-week para que cargue lazy usando loadComponent.

ANEXO A.

Una vez terminada la práctica, este proyecto debe estar en un repositorio propio, independiente del repositorio “padre” donde estés trabajando otras prácticas. La idea es desplegar este proyecto en Netlify.

- Despliegue desde GitHub.
- Configura las rutas para SPA.
- En el README.md añade: URL del repositorio y la URL del despliegue en Netlify

ANEXO B.